The Transport Layer: TCP, UDP, and Reliable Data Delivery

1. Network Layering: The OSI and TCP/IP Models

Modern computer networking is built on the principle of layered architecture. Rather than designing a single monolithic protocol that handles every aspect of communication from physical signal transmission to application semantics, engineers in the 1970s and 1980s decomposed the problem into hierarchical layers, each responsible for a narrow, well-defined function. This separation of concerns allows layers to be designed, implemented, tested, and upgraded independently, as long as the interfaces between adjacent layers remain stable.

The Open Systems Interconnection (OSI) model, standardized by the International Organization for Standardization (ISO) in 1984 as ISO/IEC 7498-1, defines seven distinct layers. From lowest to highest these are: Layer 1, the Physical layer; Layer 2, the Data Link layer; Layer 3, the Network layer; Layer 4, the Transport layer; Layer 5, the Session layer; Layer 6, the Presentation layer; and Layer 7, the Application layer.

The Physical layer is responsible for transmitting raw bit streams over a physical medium, dealing with voltages, frequencies, modulation schemes, cable types, and connector specifications. Ethernet at the physical level defines specifications such as 10BASE-T (10 Mbps over twisted pair), 100BASE-TX (Fast Ethernet), and 1000BASE-T (Gigabit Ethernet). The Data Link layer takes the raw bit stream and groups bits into frames, providing node-to-node data delivery within a single network segment. It is responsible for Media Access Control (MAC) addressing, error detection using techniques such as Cyclic Redundancy Check (CRC), and collision avoidance. Ethernet and Wi-Fi (IEEE 802.11) operate primarily at this layer. The Network layer handles logical addressing and routing, determining how packets travel from source to destination across multiple networks. The Internet Protocol (IP) — both IPv4 and IPv6 — operates at this layer. IPv4 uses 32-bit addresses, providing approximately 4.3 billion unique addresses, while IPv6 uses 128-bit addresses. The Network layer is responsible for fragmentation and reassembly when a packet is too large for a link's Maximum Transmission Unit (MTU).

The Session layer manages the opening, maintenance, and closing of sessions between applications. It provides services such as session checkpointing, which allows communication to resume from a known point after an interruption, and dialog control (half-duplex versus full-duplex). In practice, the Session layer's functionality is often absorbed into the Application layer in modern implementations. The Presentation layer handles data representation, encoding, and encryption. It ensures that data sent by the application layer of one system can be understood by the application layer of another. Functions include character encoding conversion (e.g., ASCII to EBCDIC), data compression, and data encryption/decryption. The Application layer provides network services directly to end-user applications. Protocols operating here include HTTP (port 80), HTTPS (port 443), SMTP (port 25), FTP (port 21), DNS (port 53), and many others.

The TCP/IP model, also known as the Internet model or DoD model, was developed alongside the ARPANET project in the early 1970s and is described in a series of RFC documents. It is a four-layer model. The Link layer (also called Network Access or Network Interface layer) corresponds roughly to the OSI Physical and Data Link layers combined. The Internet layer corresponds to the OSI Network layer and is where IP resides. The Transport layer corresponds directly to the OSI Transport layer. The Application layer subsumes the OSI Session, Presentation, and Application layers.

Understanding where the Transport layer sits is crucial. It sits directly above the Internet/Network layer and below the Application layer. It receives data from the Application layer above and relies on the services of the Network layer below. Critically, the Network layer provides only host-to-host delivery: it can get a packet from one machine to another, but it has no concept of which process on that machine should receive the data. The Transport layer bridges this gap by providing process-to-process delivery, identifying the specific application or service through the use of port numbers.

2. The Transport Layer's Job: Ports, Sockets, and Multiplexing

The fundamental purpose of the Transport layer is to extend the host-to-host delivery service of the Network layer into a process-to-process delivery service for applications. A modern computer runs dozens or hundreds of processes simultaneously, many of which may be communicating over the network at the same time. A web browser might have multiple tabs open, each fetching data from different servers. At the same time, a background email client, a software update manager, and a video conferencing application might all be sending and receiving network traffic. The Transport layer must sort incoming packets and deliver them to the correct process.

This is accomplished through the concept of ports. A port is a 16-bit unsigned integer, meaning it takes values in the range 0 to 65535. Ports serve as communication endpoints within a host; they allow the operating system to associate incoming traffic with a specific process or socket. The combination of an IP address and a port number forms a socket address, uniquely identifying one endpoint of a communication session.

The Internet Assigned Numbers Authority (IANA) divides the port number space into three ranges. Well-known ports span 0 through 1023 and are assigned to widely deployed protocols and services. Examples include port 20/21 for FTP (data and control respectively), port 22 for SSH (Secure Shell), port 23 for Telnet, port 25 for SMTP (email submission), port 53 for DNS, port 67/68 for DHCP (server and client), port 80 for HTTP, port 110 for POP3, port 143 for IMAP, port 443 for HTTPS, port 465 for SMTPS, port 514 for Syslog (UDP), and port 993/995 for IMAPS and POP3S respectively. On Unix-like systems, binding to ports below 1024 traditionally requires root (superuser) privileges, because those ports are considered privileged.

Registered ports span 1024 through 49151. These ports are registered with IANA to identify specific services but do not require elevated privileges to use. Examples include port 1433 for Microsoft SQL Server, port 1521 for Oracle Database, port 3306 for MySQL, port 3389 for Remote

Desktop Protocol (RDP), port 5432 for PostgreSQL, port 5900 for VNC, port 6379 for Redis, port 8080 commonly used as an alternative HTTP port, and port 27017 for MongoDB.

Dynamic or ephemeral ports span 49152 through 65535. When a client application initiates a connection, the operating system assigns it a temporary, dynamically chosen source port from this range. This port identifies the client's socket for the duration of the connection and is released when the connection closes. The Linux kernel, for example, uses the range 32768–60999 by default (configurable via the net.ipv4.ip_local_port_range kernel parameter), while many BSD systems use 49152–65535 as specified by IANA.

Multiplexing at the sender side involves gathering data from multiple application sockets, encapsulating each chunk with a header containing the appropriate source and destination port numbers, and passing the resulting segments to the Network layer. Demultiplexing at the receiver side involves examining the destination port in each arriving segment and delivering the data to the socket bound to that port. For connectionless UDP, demultiplexing uses only the destination IP address and destination port. For connection-oriented TCP, demultiplexing uses a four-tuple: source IP address, source port, destination IP address, and destination port. This means two TCP connections from different clients to the same server port are treated as separate connections and associated with separate sockets, even though both have the same destination port.

A socket is the programming abstraction that allows an application to send and receive data through the network. In the POSIX socket API, which underlies nearly all Unix-like systems and Windows networking, a socket is represented as a file descriptor. The socket system call creates a new socket; bind associates a socket with a local port; listen and accept are used by server-side TCP sockets to wait for and accept incoming connections; connect is used by TCP clients to initiate a connection; send, sendto, recv, and recvfrom are used to transfer data; and close releases the socket. The combination of (protocol, local IP, local port, remote IP, remote port) fully defines a TCP connection.

3. UDP: The User Datagram Protocol in Depth

The User Datagram Protocol (UDP) was defined by Jon Postel in RFC 768 in 1980. It is intentionally minimal. UDP provides a connectionless, unreliable, message-oriented datagram service. Every one of those adjectives carries specific technical meaning and implications.

Connectionless means that there is no handshake before data is sent and no teardown after data is sent. Each datagram is an independent packet; the sender simply constructs a datagram and fires it at the destination. There is no state maintained at either the sender or the receiver from one datagram to the next (from UDP's perspective; individual applications may track state above the transport layer). This stands in stark contrast to TCP, which maintains connection state at both endpoints throughout the life of a connection.

Unreliable means that UDP provides no delivery guarantees. A UDP datagram may be lost in transit due to network congestion, hardware failure, or queue overflow at a router. It may be duplicated, it may arrive out of order relative to other datagrams, and it may arrive corrupted

(the checksum can detect but not correct corruption). UDP will not retransmit lost datagrams, will not reorder out-of-order datagrams, and will not acknowledge receipt. If the application needs any of these services, it must implement them itself.

Message-oriented means that UDP preserves message boundaries. Each call to sendto() or send() on a UDP socket produces exactly one datagram, and each call to recvfrom() or recv() on the receiving side returns exactly one complete message. This is fundamentally different from TCP's byte-stream model, where the boundaries between individual write() calls are not preserved and a single read() may return data from multiple write() calls.

The UDP header is exactly 8 bytes (64 bits) in length, making it extremely compact. It consists of four fields, each 16 bits wide. The Source Port field (bits 0–15) identifies the port of the sending process. This field is optional; if the sending process does not wish to receive replies, it may be set to zero. The Destination Port field (bits 16–31) identifies the port of the intended recipient process and is always required. The Length field (bits 32–47) specifies the total length of the UDP datagram in bytes, including both the 8-byte header and the data payload. The minimum value is 8 (an empty datagram with no payload). The maximum value is 65535 bytes, but since the total IP packet size is also bounded by 65535 bytes, and the IP header is at least 20 bytes, the maximum UDP payload is 65507 bytes. The Checksum field (bits 48–63) provides a check over a pseudo-header (comprising source IP, destination IP, a zero byte, the protocol number 17, and the UDP length), the UDP header, and the UDP data. For IPv4, the checksum is optional (a value of zero means the checksum was not computed); for IPv6, the checksum is mandatory because IPv6 does not include a header checksum.

The UDP checksum uses the same one's complement addition algorithm as the TCP checksum. The sender computes the checksum by summing all 16-bit words of the pseudo-header, UDP header, and data (padding with a zero byte if the data length is odd), then taking the one's complement of the result. The receiver recomputes the checksum; if the result is all ones (0xFFFF in one's complement), no error was detected.

Because UDP has no congestion control, a UDP sender can inject data into the network at any rate it chooses, limited only by the application and the network interface. This makes UDP attractive for applications that are latency-sensitive and can tolerate some data loss, but it also means UDP applications can contribute to network congestion. This is why designing well-behaved UDP applications that perform their own congestion control (as modern QUIC does) is considered important for network health.

The use cases for UDP fall into several categories. Real-time multimedia streaming, including VoIP (Voice over IP) and video conferencing, favors UDP because the latency introduced by TCP's retransmission mechanism would render the application unusable: a retransmitted audio packet that arrives 200 milliseconds late is worse than useless. Protocols such as RTP (Real-time Transport Protocol) run over UDP. Online gaming uses UDP for similar reasons: game state updates must be delivered with minimal latency, and if a position update is lost, a more recent update will correct the state anyway. DNS (Domain Name System) uses UDP on port 53 for most

queries and responses because DNS messages are small (typically fitting within a single datagram), the protocol's own request/response model handles timeouts and retries at the application layer, and the overhead of establishing a TCP connection would add unacceptable latency to every name lookup. DHCP (Dynamic Host Configuration Protocol) uses UDP on ports 67 (server) and 68 (client) because at the time a host requests an IP address, it does not yet have one, making connection-oriented communication impossible. SNMP (Simple Network Management Protocol) uses UDP ports 161 (agent) and 162 (manager/trap receiver) because polling and traps are often acceptable to lose occasionally. Streaming media delivery using protocols like RTP/RTCP leverages UDP, with RTCP providing out-of-band statistics to allow adaptive bitrate control. TFTP (Trivial File Transfer Protocol) uses UDP on port 69 with its own simple acknowledgment scheme built on top. NTP (Network Time Protocol) uses UDP on port 123.

4. TCP: Connection-Oriented, Reliable, In-Order Byte Streams

The Transmission Control Protocol (TCP) was first formally specified in RFC 793 by Jon Postel in September 1981, building on earlier work in RFC 675 by Vint Cerf, Yogen Dalal, and Carl Sunshine in 1974. TCP provides a fundamentally richer set of services than UDP, all at the cost of additional complexity, overhead, and latency. The four defining characteristics of TCP are that it is connection-oriented, reliable, in-order, and a byte stream (full-duplex).

Connection-oriented means that before any data can be exchanged, the two endpoints must go through a connection establishment procedure — the three-way handshake — to agree on initial sequence numbers, exchange capabilities, and allocate resources. Similarly, there is a formal connection termination sequence. This is analogous to a telephone call: you dial, the other party answers, you converse, and then you both say goodbye and hang up.

Reliable means that TCP guarantees delivery of all data and acknowledgment of receipt. Every byte sent is tracked with a sequence number. The receiver sends acknowledgments confirming receipt. If an acknowledgment is not received within a timeout period, the sender retransmits the data. From the application's perspective, data written to a TCP socket will either be delivered to the remote application exactly once and in order, or the connection will be aborted with an error.

In-order delivery means that even if segments arrive out of order at the receiver (which can happen because different IP packets may take different paths through the network), TCP will buffer and reorder them before delivering them to the application layer.

Byte-stream means that TCP treats data as a continuous stream of bytes, not as a sequence of discrete messages. Application message boundaries are not preserved. If an application writes 100 bytes and then 200 bytes to a TCP socket, the receiving application might read 300 bytes at once, or 50 bytes six times, or any other combination. Applications that need message framing must implement it themselves, typically by prepending each message with its length or using delimiter characters.

Full-duplex means that data can flow in both directions simultaneously on a single TCP

connection. Each direction has its own independent sequence number space and flow control window. This allows, for example, a web server to send a large response while simultaneously receiving a subsequent request from the same client without either direction being blocked by the other.

5. The TCP Segment Header

The TCP segment header has a minimum size of 20 bytes (160 bits) and can be extended to up to 60 bytes with options. The header contains a rich set of fields that enable all of TCP's complex functionality.

The Source Port field (16 bits) identifies the sending process's port number. The Destination Port field (16 bits) identifies the receiving process's port number. Together with the IP addresses, these four values form the connection four-tuple.

The Sequence Number field (32 bits) serves a dual purpose. If the SYN flag is set, this field contains the Initial Sequence Number (ISN) chosen by the sender. If the SYN flag is not set, this field contains the sequence number of the first byte of data in the segment's payload. Sequence numbers wrap around modulo 2^32 (approximately 4.3 billion). Choosing sequence numbers that wrap around without ambiguity is one of the reasons ISNs are chosen pseudo-randomly rather than always starting at zero.

The Acknowledgment Number field (32 bits) is valid only when the ACK flag is set. It contains the sequence number of the next byte that the sender of the segment expects to receive from the other side. This is a cumulative acknowledgment: an acknowledgment number of N means all bytes with sequence numbers less than N have been received successfully and in order.

The Data Offset field (4 bits), also called the Header Length field, specifies the length of the TCP header in 32-bit words. The minimum value is 5 (for the 20-byte minimum header with no options) and the maximum value is 15 (for a 60-byte header). This field is necessary because the TCP options field has variable length.

The Reserved field (3 bits) is reserved for future use and should be set to zero.

The Control Flags field (9 bits in modern TCP, though historically 6 bits) contains individual single-bit flags. The most important flags are: URG — indicates that the Urgent Pointer field is significant and there is urgent data in this segment; ACK — indicates that the Acknowledgment Number field is valid; PSH (Push) — requests that buffered data be pushed to the receiving application immediately rather than being held until the buffer fills; RST (Reset) — requests an immediate abort of the connection, used when an error has occurred or when a segment arrives for a non-existent connection; SYN — used during connection establishment to synchronize sequence numbers; FIN — indicates that the sender has no more data to send and wishes to close its half of the connection. Additional flags added in later RFCs include ECE (ECN-Echo) and CWR (Congestion Window Reduced), which support Explicit Congestion Notification (ECN) as defined in RFC 3168.

The Window Size field (16 bits) specifies the size of the receive buffer that the sender of the segment is willing to accept, measured in bytes. This is the basis for TCP's flow control mechanism. A value of 0 means the receiver's buffer is full and the sender must pause. The maximum window size representable in this field without options is 65535 bytes. The TCP Window Scale option (RFC 7323) allows this to be scaled up by a factor of up to 2^14, enabling window sizes of up to 1 gigabyte, which is necessary for efficient use of high-bandwidth, high-latency networks.

The Checksum field (16 bits) is computed over a pseudo-header (source IP, destination IP, zero byte, protocol number 6, TCP segment length), the TCP header, and the TCP data payload. Unlike the UDP checksum, the TCP checksum is mandatory.

The Urgent Pointer field (16 bits) is valid only when the URG flag is set. It indicates the offset from the sequence number of the last byte of urgent data. Urgent data (also called out-of-band data) is infrequently used in modern protocols.

The Options field is variable in length (0 to 40 bytes, in multiples of 4 bytes due to alignment requirements, with padding added if necessary). Commonly used TCP options include: Maximum Segment Size (MSS) — exchanged during the three-way handshake, it specifies the largest segment the sender is willing to receive; Window Scale — also exchanged during the handshake, it defines the left-shift factor for the advertised window; SACK Permitted and SACK (Selective Acknowledgment) — the SACK Permitted option, exchanged during handshake, indicates support for selective acknowledgments, and the SACK option carries acknowledgment blocks specifying which out-of-order segments have been received; Timestamps — the TCP Timestamps option (RFC 7323) allows accurate RTT measurement and provides protection against wrapped sequence numbers (PAWS — Protection Against Wrapped Sequence numbers); No-Operation (NOP) — a one-byte option used for padding; End of Option List (EOL) — a one-byte option indicating no more options.

6. Connection Establishment and Termination: The Three-Way Handshake and Four-Way Teardown

TCP connection establishment uses a three-way handshake to ensure both parties are ready to communicate and to synchronize their initial sequence numbers. The process works as follows.

In the first step, the client sends a segment with the SYN flag set and no ACK flag. This segment carries the client's Initial Sequence Number (ISN), call it x, in the Sequence Number field. The client enters the SYN_SENT state. The ISN is not simply zero or one; it is chosen using a clock-based or pseudo-random algorithm to reduce the risk of old duplicate segments from a previous connection being mistaken for new ones. RFC 6528 specifies that ISNs should be generated using a cryptographic hash function to prevent ISN prediction attacks.

In the second step, the server receives the SYN and responds with a segment that has both SYN and ACK flags set. The Sequence Number field of this segment contains the server's own ISN, call it y. The Acknowledgment Number field contains x + 1, acknowledging the client's SYN (which

consumes one sequence number). The server enters the SYN_RECEIVED state.

In the third step, the client receives the SYN-ACK and sends a final ACK segment. The Acknowledgment Number field is $y + 1$, acknowledging the server's SYN. The Sequence Number field is $x + 1$. Both endpoints are now in the ESTABLISHED state, and data transfer can begin. The three-way handshake introduces at minimum one round-trip time (RTT) of latency before data can be sent.

SYN cookies are a technique to protect servers from SYN flood attacks (a type of denial-of-service attack where an attacker sends a large number of SYN segments without completing the handshake). When SYN cookies are enabled, the server does not allocate any resources when it receives a SYN; instead, it encodes state information (client IP, client port, server port, timestamp) into the ISN it sends back. If the client completes the handshake with a valid ACK, the server decodes the connection state from the acknowledgment number. This prevents resource exhaustion but forfeits the ability to use TCP options negotiated during the handshake.

Connection termination is more complex than establishment. Because TCP is full-duplex, each direction must be closed independently. The standard four-way termination sequence works as follows. Either side may initiate termination. Suppose the client initiates. First, the client sends a FIN segment and enters the FIN_WAIT_1 state. Second, the server sends an ACK for the FIN and enters the CLOSE_WAIT state. The client enters FIN_WAIT_2 state upon receiving this ACK. The server can continue sending data at this point (half-close state). Third, when the server is done sending, it sends its own FIN segment and enters the LAST_ACK state. Fourth, the client receives the server's FIN, sends an ACK, and enters the TIME_WAIT state. The server receives the ACK and enters the CLOSED state.

The TIME_WAIT state is significant. The endpoint that actively closes the connection (sends the first FIN) enters TIME_WAIT and remains there for 2 * MSL (Maximum Segment Lifetime) seconds before the connection resources are released. MSL is typically defined as 60 seconds (making TIME_WAIT last 120 seconds, or 2 minutes) though it varies by implementation. The purpose of TIME_WAIT is twofold: first, it ensures the final ACK has time to reach the server (if the server retransmits its FIN, the client can re-send the ACK from TIME_WAIT); second, it prevents stale segments from a previous incarnation of the connection from being mistaken for segments from a new connection on the same four-tuple. TIME_WAIT can cause issues on servers that open and close many short-lived connections rapidly, as the port space may become exhausted with TIME_WAIT sockets. The SO_REUSEADDR and SO_REUSEPORT socket options, as well as TCP_QUICKACK, are used to mitigate this in practice.

A connection can also be terminated abruptly with a RST (Reset) segment. A RST causes immediate teardown with no TIME_WAIT delay and no graceful drain of buffered data. RST segments are sent in response to segments that arrive for non-existent connections, as an error response, or when an application calls close() with the SO_LINGER option set to zero.

7. Reliable Delivery: Sequence Numbers, Acknowledgments, and

Retransmission

TCP's reliability mechanism is built on a foundation of sequence numbers and acknowledgments. Every byte of data in a TCP stream is assigned a unique sequence number. When the sender places data into a segment, the Sequence Number field identifies the position of the first byte of that data within the overall stream. The receiver uses these sequence numbers to detect missing data, to reorder out-of-order segments, and to avoid delivering duplicate data to the application.

Acknowledgments in TCP are cumulative. When the receiver sends an ACK with an acknowledgment number of N, it means that all bytes with sequence numbers from the start of the connection up to but not including N have been received. If a segment is missing from the middle of the stream, the receiver cannot advance the acknowledgment number past the gap, even if it has received later segments. This cumulative ACK mechanism is simple and efficient but has a limitation: the sender cannot tell from a single ACK number exactly which segments were received and which were not.

The Retransmission Timeout (RTO) is the time the sender waits after sending a segment before concluding that it has been lost and retransmitting it. Setting the RTO correctly is critical: too short and segments are retransmitted unnecessarily (wasting bandwidth and increasing congestion); too long and the sender waits excessively after a genuine loss (reducing throughput). RFC 6298 specifies the standard algorithm for computing the RTO.

The sender maintains an estimate of the round-trip time (RTT), which is the time from sending a segment to receiving its acknowledgment. The smoothed RTT (SRTT) is an exponentially weighted moving average: SRTT = (1 - alpha) * SRTT + alpha * RTT_sample, where alpha is typically 1/8 (0.125). The sender also maintains an RTT variation estimate (RTTVAR): RTTVAR = (1 - beta) * RTTVAR + beta * |SRTT - RTT_sample|, where beta is typically 1/4 (0.25). The RTO is then computed as: RTO = SRTT + max(G, 4 * RTTVAR), where G is a clock granularity factor (typically 0.1 seconds). This algorithm was proposed by Van Jacobson in 1988 and is often called the Jacobson/Karels algorithm. The RTO has a minimum value of 1 second per RFC 6298 and typically a maximum value of 60 seconds. When a retransmission timeout fires, the RTO is doubled (binary exponential backoff) up to the maximum, and a segment is retransmitted.

Karn's algorithm addresses a subtle problem: when a retransmitted segment is acknowledged, the sender cannot tell whether the ACK is for the original transmission or the retransmission. If the ACK is for the retransmission, using the measured RTT sample would underestimate the actual RTT; if it is for the original, it would overestimate. Karn's algorithm solves this by not updating the SRTT estimate from retransmitted segments. Additionally, the RTO is not updated until an ACK is received for a non-retransmitted segment.

Fast Retransmit is an optimization that allows TCP to recover from a lost segment without waiting for the full RTO to expire. When the receiver receives an out-of-order segment — that is, a segment whose sequence number is higher than the next expected — it sends an immediate duplicate ACK (a segment with the same ACK number as the most recent valid ACK). This signals to the sender that a gap has been detected. Per RFC 5681, when the sender receives three

consecutive duplicate ACKs (for a total of four identical ACK segments), it infers that the segment immediately following the acknowledged data has been lost and retransmits it immediately, before the RTO expires. This is called fast retransmit. The threshold of three duplicate ACKs is a heuristic that balances sensitivity (detecting loss quickly) against false positives (avoiding retransmissions caused by mere reordering).

Selective Acknowledgment (SACK), defined in RFC 2018, extends TCP's acknowledgment capabilities. With SACK, the receiver can inform the sender not only of the highest contiguous byte received (the standard cumulative ACK) but also of non-contiguous blocks of data that have been received. The SACK option in the TCP header carries up to four SACK blocks, each specifying the left and right edges (sequence numbers) of a range of received data. Armed with this information, the sender can retransmit only the specific segments that are missing rather than retransmitting all data after the first loss. This significantly improves performance in environments where multiple segments are lost from a single window of data. SACK must be negotiated during the three-way handshake using the SACK Permitted option; if either side omits this option, SACK cannot be used. Forward Acknowledgment (FACK) is a related technique that uses SACK information to make more precise congestion window adjustments.

DSACK, or Duplicate SACK (RFC 2883), extends SACK to also report segments that have been received more than once (duplicates). This allows the sender to distinguish between segment loss (which caused a retransmission that arrived after the original) and spurious retransmissions (caused by false timeout or reordering). With D-SACK information, the sender can adjust its retransmission behavior to avoid unnecessarily retransmitting in the future.

8. Flow Control: The Sliding Window and Receiver Buffer Management

Flow control is the mechanism by which a TCP receiver prevents a fast sender from overwhelming it with more data than it can process or buffer. It is distinct from congestion control: flow control is about the receiver's capacity, while congestion control is about the network's capacity. The two mechanisms operate simultaneously and their effects are multiplicative.

The central mechanism of TCP flow control is the sliding window, specifically the receiver advertised window (also called rwnd). Each TCP segment contains a 16-bit Window Size field, which the receiver fills with the amount of free space currently available in its receive buffer. The sender may not have more than rwnd bytes of unacknowledged data outstanding (in flight) at any time. As the receiver reads data from its buffer and delivers it to the application, the buffer space frees up, and the receiver sends ACK segments with a larger window value, allowing the sender to transmit more data. This creates a feedback loop that naturally matches the sending rate to the receiving application's consumption rate.

The receiver's buffer size is an operating system parameter. On Linux, the default TCP receive buffer size is controlled by net.core.rmem_default, with a maximum set by net.core.rmem_max.

The actual buffer is divided into three zones: one for memory overhead, one for application data not yet read, and one for out-of-order data being held. The kernel can automatically scale receive buffers (TCP autotuning) within these limits based on observed RTT and bandwidth.

A zero-window condition (also called window closed or window probe) occurs when the receiver's buffer fills completely and it advertises a window of 0. The sender must stop transmitting data. However, if the sender simply waited indefinitely, it might never hear that the window has reopened (if the window update ACK from the receiver were lost). To prevent this deadlock, TCP uses zero-window probes: when the sender's window is closed, it periodically sends zero-window probe segments (segments with one byte of data or no data, depending on implementation) to prompt the receiver to re-advertise its window size. The interval between probes uses exponential backoff.

Silly Window Syndrome (SWS) is a pathological condition that occurs when either the receiver advertises a very small window or the sender transmits very small segments, leading to poor network efficiency — the overhead of TCP/IP headers dominates the payload. The syndrome was named by Clark (1982). There are two complementary solutions. The receiver-side SWS avoidance rule (RFC 1122) states that the receiver should not advertise a window increment until the available space has grown to at least one maximum segment size (MSS) or half the total receive buffer, whichever is smaller. This prevents the window from being opened in small increments. The sender-side solution is Nagle's algorithm.

Nagle's algorithm, proposed by John Nagle in RFC 896 (1984) and included as a recommended feature in RFC 1122, prevents a TCP sender from sending many small segments by requiring that if there is unacknowledged data, new small data should be buffered and aggregated until either enough data accumulates to fill a full MSS-sized segment or all previously sent data has been acknowledged. More precisely, Nagle's algorithm states: a sender can always send a full-sized segment; if there is no outstanding unacknowledged data, send any amount of data; otherwise, buffer data until the send buffer accumulates enough for a full segment. This algorithm dramatically improves efficiency for interactive applications that send one keystroke at a time (like Telnet or SSH), converting what would have been 41-byte segments (1 byte data + 40 bytes IP/TCP header) into fewer, larger segments. However, Nagle's algorithm can introduce latency in interactive applications. For this reason, the TCP_NODELAY socket option can be set to disable Nagle's algorithm, which is common for latency-sensitive applications such as gaming clients, financial trading systems, and some database clients.

The TCP Window Scale option (part of RFC 7323, formerly RFC 1323) extends the effective window size beyond the 65535-byte limit imposed by the 16-bit Window Size field. During the three-way handshake, each side can include a Window Scale option specifying a shift count (0 to 14). The actual window size is then the value in the Window Size field left-shifted by the scale factor. With a scale factor of 14, the maximum window size is 65535 * 2^14 = 1,073,725,440 bytes, or approximately 1 GB. This is essential for high-speed, long-distance connections (often called elephant flows) where the bandwidth-delay product is large. For example, on a 1 Gbps link with a 100 ms RTT, the bandwidth-delay product is 1,000,000,000 * 0.1 / 8 = 12.5 MB; without

window scaling, the maximum window of 65535 bytes would cap throughput at about 524 Kbps, a factor of 1900 below the link capacity.

9. Congestion Control: Slow Start, AIMD, and the Congestion Window

While flow control prevents the sender from overwhelming the receiver, congestion control prevents the collective sending rates of all TCP flows from overwhelming the network — specifically, its bottleneck links and router queues. When the network is congested, routers drop packets (or mark them in the ECN variant), causing retransmissions that waste bandwidth and further worsen congestion. Without congestion control, TCP could trigger a congestion collapse, a phenomenon observed on the ARPANET in 1986 where throughput dropped by a factor of one thousand due to TCP implementations sending at full speed and immediately retransmitting dropped packets in a destructive feedback loop.

Van Jacobson introduced the key TCP congestion control algorithms in a landmark 1988 paper and in RFC 2001, later superseded by RFC 5681. TCP congestion control maintains a congestion window (cwnd), a sender-side limit on the number of unacknowledged bytes that may be in flight. The effective window (the actual limit on outstanding data) is the minimum of cwnd and rwnd (the receiver's advertised window). TCP also maintains a slow start threshold (ssthresh), which governs the transition between two modes of operation.

Slow Start is TCP's initial phase. Despite the name, slow start is actually an exponential growth phase. It begins with cwnd = 1 MSS (or, in modern systems, cwnd = 10 MSS per RFC 6928 as the initial window). For every ACK received, cwnd is increased by one MSS. Since the number of ACKs received per round trip is approximately equal to the current cwnd, this results in a doubling of cwnd every RTT — exponential growth. Slow start continues until cwnd reaches ssthresh, at which point TCP transitions to congestion avoidance. Slow start also applies after a retransmission timeout, where both cwnd is reset to 1 MSS (or 1 segment in older implementations) and ssthresh is updated.

Congestion Avoidance is the steady-state operating mode. Instead of doubling cwnd each RTT, TCP increases cwnd by approximately one MSS per RTT, regardless of the current cwnd value. The standard implementation does this by increasing cwnd by MSS * (MSS / cwnd) for each incoming ACK, which has the effect of increasing cwnd by one MSS per RTT. This linear growth is the "additive increase" part of AIMD. Congestion avoidance continues until congestion is detected.

Congestion detection in TCP occurs in two ways. A timeout (the RTO fires) indicates severe congestion. In TCP Tahoe (one of the earliest implementations, named after the Lake Tahoe algorithm workshop where it was refined), any packet loss event — whether a timeout or three duplicate ACKs — causes ssthresh to be set to max(cwnd/2, 2*MSS) and cwnd to be reset to 1 MSS, restarting slow start from scratch. In TCP Reno (a subsequent refinement), timeout is handled the same as Tahoe (cwnd reset to 1 MSS), but three duplicate ACKs (indicating fast retransmit) trigger a different response: ssthresh is set to max(cwnd/2, 2*MSS), and cwnd is set to

ssthresh + 3*MSS. This is the "multiplicative decrease" part of AIMD. TCP Reno then enters a state called Fast Recovery.

Fast Recovery (RFC 5681) is specific to TCP Reno and its successors. After entering fast recovery (triggered by three duplicate ACKs), the sender has already set ssthresh = cwnd/2 and cwnd = ssthresh + 3*MSS. For each additional duplicate ACK received while in fast recovery, cwnd is increased by one MSS (reflecting that another segment has left the network). This inflated cwnd allows the sender to continue transmitting new segments during recovery. When a new, non-duplicate ACK arrives (meaning the retransmitted segment was delivered), cwnd is deflated back to ssthresh and the sender returns to congestion avoidance mode. This approach is generally more efficient than Tahoe's approach of resetting to slow start, because it continues sending during recovery rather than stalling for an RTT.

The AIMD (Additive Increase, Multiplicative Decrease) principle gives TCP its characteristic sawtooth cwnd profile. When the network is not congested, cwnd grows linearly by one MSS per RTT. When congestion is detected, cwnd is cut in half. This behavior has been mathematically proven to converge to a fair sharing of bandwidth among competing flows: if n flows compete on a bottleneck link, each should receive approximately 1/n of the link capacity in steady state, assuming similar RTTs. This property is called TCP fairness, though in practice differences in RTT (flows with shorter RTTs get higher throughput) and differences in protocol (UDP flows do not back off) complicate the fairness picture.

TCP New Reno (RFC 6582) is an improvement over Reno that handles the case where multiple segments are lost from a single window of data more gracefully. Standard Reno can only detect one loss per RTT in the absence of SACK, because after retransmitting the first lost segment, the next new ACK (a partial ACK, acknowledging only up to the retransmitted segment) exits fast recovery and returns to congestion avoidance before detecting that additional segments from the same window were also lost. New Reno remains in fast recovery mode and retransmits the next lost segment when it receives a partial ACK, continuing until all losses from the window have been recovered.

TCP CUBIC (RFC 8312) is the default congestion control algorithm in Linux since version 2.6.19. Instead of Reno's linear increase, CUBIC uses a cubic function of the time elapsed since the last congestion event to determine cwnd, allowing faster recovery to the previous maximum window size without being overly aggressive near the congestion point. CUBIC's growth is independent of RTT, making it more fair between flows with different RTTs compared to Reno-based algorithms.

Explicit Congestion Notification (ECN, RFC 3168) allows routers to signal impending congestion to TCP endpoints without dropping packets. When a router's queue is building up, it can set ECN bits (CE — Congestion Experienced) in the IP header of passing packets rather than dropping them. The receiver echoes this signal back to the sender using the ECE flag in the TCP header. The sender responds by reducing cwnd exactly as if a packet had been dropped. ECN requires support from both endpoints (negotiated using the ECE and CWR flags during the handshake) and from the routers on the path. Because ECN avoids the retransmission caused by packet dropping,

it can improve throughput and reduce latency during periods of congestion.

10. TCP vs. UDP: A Systematic Comparison

TCP and UDP represent two fundamentally different design philosophies for the Transport layer, each with clear advantages in appropriate contexts. Understanding the trade-offs is essential for selecting the right protocol for a given application.

Reliability is the most fundamental difference. TCP guarantees that data will be delivered, that it will not be duplicated, and that it will be in order. This comes from the sequence number, acknowledgment, and retransmission mechanisms described above. UDP provides no such guarantees. Datagrams may be lost, duplicated, or delivered out of order, and UDP does nothing about any of these conditions. Applications that need reliability over UDP must implement it themselves.

Ordering guarantees differ similarly. TCP's in-order delivery means the application always receives data in the order it was sent. UDP preserves no ordering; each datagram is routed independently and may follow a different path. Multiple datagrams sent in order may arrive in any order, and the application must handle reordering if it matters.

Connection state is required by TCP but not by UDP. TCP requires a three-way handshake (minimum 1 RTT of latency) before data can flow, and a four-way teardown at the end. The kernel must allocate memory for connection state — including send and receive buffers, sequence number tracking, congestion window variables, and more — for each established TCP connection. A server handling 100,000 simultaneous TCP connections must maintain state for all of them. UDP requires no per-connection state in the transport layer, which is why DNS servers and other high-query-rate services often prefer it.

Header overhead is 20 bytes minimum for TCP and 8 bytes for UDP. For applications sending tiny payloads (like a 12-byte DNS query), the TCP header overhead is more than double the useful data. For applications sending large payloads, the 12-byte header difference is negligible as a fraction of total packet size.

Throughput and latency trade-offs depend strongly on the application. For bulk data transfer over a reliable, low-loss network (file transfer, web pages, database queries), TCP's overhead is acceptable and its reliability guarantees simplify application design enormously. For real-time applications with strict latency budgets (VoIP, live video, gaming), even a single retransmission can delay data by tens of milliseconds and destroy the application experience, making UDP preferable despite its unreliability.

Congestion control is built into TCP. TCP will slow down when the network is congested, protecting both itself and other flows. UDP has no such mechanism; a UDP sender at full speed will not back off under congestion, potentially crowding out TCP flows and other UDP flows. This is why well-designed UDP-based applications (WebRTC, QUIC) implement their own congestion control at the application or transport-within-UDP level.

Message boundaries: UDP preserves message boundaries (each send produces exactly one datagram, each receive delivers exactly one datagram). TCP's byte-stream model provides no message boundaries. For protocols that need framing (discrete messages of varying sizes), UDP is more natural, though TCP framing can be added by the application.

Broadcast and multicast: UDP supports broadcast (sending to all hosts on a subnet) and multicast (sending to a group of hosts subscribed to a multicast group address). TCP is strictly unicast — one sender to one receiver per connection. This makes UDP the only choice for protocols that need to reach multiple recipients with a single transmission, such as multicast video distribution protocols and some network discovery protocols.

The rule of thumb for choosing between TCP and UDP is: use TCP when correctness matters more than latency (file transfer, email, web, database), and use UDP when latency matters more than correctness (live audio/video, gaming) or when the protocol handles its own reliability (DNS, DHCP, SNMP with their own retry logic). Many modern high-performance applications have concluded that neither pure TCP nor pure UDP meets their needs and have built custom transport mechanisms on top of UDP.

11. Modern Developments: QUIC and HTTP/3

QUIC (Quick UDP Internet Connections) is a transport protocol originally developed by Google in 2012–2013 and standardized by the IETF in RFC 9000 (May 2021). QUIC runs on top of UDP (typically on port 443) rather than being a standalone protocol like TCP. This design choice was deliberate and strategic: because TCP is implemented in operating system kernels, updating it requires OS updates, which happen slowly. By building on UDP, which is available on every OS, QUIC can be deployed and updated as part of application software, including web browsers. QUIC is the transport protocol underlying HTTP/3 (RFC 9114).

QUIC addresses several fundamental limitations of TCP over HTTPS. The first is head-of-line blocking. HTTP/2 over TLS-over-TCP introduced multiplexing of multiple HTTP streams over a single TCP connection. However, because TCP provides a single ordered byte stream, if a TCP segment is lost, all streams must stall while waiting for retransmission, even if the lost segment only contained data for one of the streams. QUIC implements multiplexing at the transport layer with independent stream abstractions; a lost QUIC packet only stalls the stream(s) to which it belonged, not all streams. This is a fundamental architectural improvement.

The second major advantage is connection establishment latency. Establishing an HTTPS connection over TCP requires: 1 RTT for the TCP three-way handshake, followed by 1 RTT for the TLS 1.3 handshake (or 2 RTTs for TLS 1.2), totaling 2–3 RTTs before the first byte of application data can be sent. QUIC combines transport and cryptographic handshake into a single 1-RTT exchange. Furthermore, QUIC supports 0-RTT resumption: if a client has previously connected to a server, it can send data on the very first packet, before any round-trip, using a pre-shared session ticket. This is especially valuable for mobile users and high-latency networks.

Connection migration is another QUIC feature. TCP connections are identified by the four-tuple

(source IP, source port, destination IP, destination port). When a mobile device switches from a Wi-Fi network to a cellular network, its IP address changes, and any existing TCP connection breaks and must be re-established. QUIC uses 64-bit connection IDs that are independent of IP addresses and port numbers. When the underlying network path changes, a QUIC connection can migrate to the new address without interruption, as long as the cryptographic state is maintained. This is a significant user experience improvement for mobile applications.

QUIC encrypts everything except the outermost UDP header. Not just the payload, as TLS over TCP does, but also transport-layer metadata including stream IDs, acknowledgment information, and flow control data. This prevents middleboxes (NATs, firewalls, load balancers, transparent proxies) from inspecting or modifying transport-layer state, which has historically allowed middleboxes to interfere with TCP extensions. The trade-off is that QUIC connections are harder to inspect for network management purposes.

QUIC implements its own congestion control (defaulting to CUBIC or BBR), flow control (per-stream and per-connection windows), loss detection (with improved RTT estimation compared to TCP), and stream multiplexing. QUIC also supports datagrams (RFC 9221) for applications that need unreliable delivery (like games and real-time media) while still benefiting from QUIC's connection migration and encryption.

HTTP/3 (RFC 9114) is the third major version of the Hypertext Transfer Protocol, running over QUIC instead of TCP. HTTP/3 maps HTTP semantics (methods, headers, bodies, streams) directly to QUIC streams. The QPACK header compression scheme (RFC 9204) replaces HPACK from HTTP/2, redesigned to work with QUIC's independent streams. QUIC's stream model maps cleanly to HTTP's request/response model: each HTTP/3 request/response pair uses a separate QUIC stream.

TLS (Transport Layer Security) over TCP is the incumbent security mechanism for TCP connections. TLS 1.3, standardized in RFC 8446 (2018), is the current version. TLS 1.3 reduced the handshake from 2 RTTs (in TLS 1.2) to 1 RTT, and supports 0-RTT resumption. TLS provides authentication (via X.509 certificates), confidentiality (via symmetric encryption, typically AES-GCM or ChaCha20-Poly1305), and integrity (via HMAC or AEAD). TLS operates above the Transport layer, encapsulating the TCP byte stream, though its handshake interacts closely with the transport for timing reasons.

TCP Fast Open (TFO, RFC 7413) is a TCP extension that allows the client to send data in the SYN packet during connection establishment (or re-establishment), reducing the round-trip latency for short TCP connections. Like QUIC's 0-RTT, TFO uses a cookie exchanged during the first connection to authenticate subsequent 0-RTT data. TFO has seen limited adoption due to issues with middleboxes that drop SYN packets containing data and security concerns about 0-RTT data replay.

Multipath TCP (MPTCP, RFC 8684) extends TCP to use multiple network paths simultaneously for a single connection. A smartphone connected to both Wi-Fi and LTE could use MPTCP to send data over both simultaneously, increasing bandwidth and resilience. MPTCP uses the standard

TCP connection model at the application interface but manages multiple subflows underneath. Apple has used MPTCP for Siri since iOS 7.

12. Sockets and the Client–Server Model in Practice

The client–server model is the dominant pattern for networked applications. A server is a process that listens on a well-known port, waiting for clients to connect or send requests. A client is a process that initiates communication, typically using a dynamically assigned ephemeral port as its source port.

In the POSIX socket API, a TCP server creates and manages connections in a specific sequence. The server first creates a socket using the socket() system call with parameters AF_INET (or AF_INET6 for IPv6) for the address family, SOCK_STREAM for the socket type (indicating a byte-stream, connection-oriented socket for TCP), and IPPROTO_TCP for the protocol. It then calls bind() to associate the socket with a specific local IP address (or INADDR_ANY to listen on all interfaces) and port number. The server calls listen() to mark the socket as passive and to specify the backlog — the maximum number of pending connections that may queue up while the server is busy. The accept() system call blocks until a new client connects, at which point it returns a new socket descriptor representing that specific client connection. The original listening socket remains available to accept further connections. The server reads data using recv() or read() and writes data using send() or write(). Finally, close() tears down the connection (initiating the four-way FIN exchange).

A TCP client uses socket() to create a socket, optionally calls bind() to specify a source port (usually omitted, allowing the OS to assign an ephemeral port automatically), and then calls connect() with the server's IP address and port number. The connect() call triggers the three-way handshake; it blocks until the handshake completes (or fails with an error such as ECONNREFUSED if the server has no listening socket on that port, or ETIMEDOUT if no response is received). After connect() returns successfully, the client uses send() and recv() for data exchange.

A UDP server creates a socket with SOCK_DGRAM instead of SOCK_STREAM, calls bind() to associate it with a port, and then uses recvfrom() (which also returns the sender's address) and sendto() (which requires specifying the destination address with each call) to exchange datagrams. There is no listen(), accept(), or connect() required. A UDP client can use connect() to associate a remote address with the socket (which merely stores the address for use with subsequent send()/recv() calls and enables the kernel to filter incoming datagrams), but this is optional.

The backlog parameter to listen() specifies the length of the kernel's queue of completed (three-way handshake finished) connections waiting to be accepted by the server's accept() call. In Linux, there is actually a separate queue for incomplete connections (SYNs received but handshake not yet complete). The maximum backlog is bounded by the net.core.somaxconn kernel parameter (default 128 in older Linux, 4096 in more recent versions). When the backlog

queue is full, incoming SYNs may be dropped. High-performance servers may tune this value upward and use multiple threads or processes calling accept() concurrently (the pre-forking model) or use asynchronous I/O (epoll on Linux, kqueue on BSD/macOS) to handle thousands of concurrent connections efficiently.

The SO_REUSEADDR socket option allows a server to bind to a port that is in TIME_WAIT from a previous connection on that port, which is essential for servers that need to restart quickly after a crash without waiting up to 4 minutes for TIME_WAIT to expire. SO_REUSEPORT (Linux 3.9+, BSD) allows multiple sockets to bind to the same address and port, with the kernel distributing incoming connections (TCP) or datagrams (UDP) among them. This allows multiple worker processes or threads to accept connections in parallel without contention on a single accept() call.

Non-blocking sockets allow applications to manage multiple connections in a single thread using I/O multiplexing. The classic select() system call allows a process to wait until one or more file descriptors are ready for reading, writing, or have encountered an error, with a timeout. poll() provides similar functionality with a more convenient interface. On Linux, epoll (introduced in Linux 2.5.44) provides O(1) event notification regardless of the number of monitored file descriptors, making it far more efficient than select() or poll() for servers managing thousands of simultaneous connections. The epoll_create(), epoll_ctl(), and epoll_wait() system calls form the epoll API. Epoll supports level-triggered (LT) and edge-triggered (ET) modes, where ET mode only notifies when the state changes (e.g., new data arrived) rather than as long as data is available, requiring careful buffering to avoid missing events.

The Internet Protocol version 6 (IPv6) transport layer behavior is essentially identical to IPv4. TCP and UDP over IPv6 use the same port mechanism, three-way handshake, congestion control algorithms, and socket API (with AF_INET6 and sockaddr_in6 structures instead of AF_INET and sockaddr_in). One difference is that IPv6 removes the NAT (Network Address Translation) middlebox common in IPv4 networks, which simplifies peer-to-peer connectivity but requires that congestion control and flow control work correctly over the public internet without the asymmetries introduced by NAT.

TCP performance tuning in practice involves adjusting multiple kernel parameters. On Linux, net.ipv4.tcp_rmem and net.ipv4.tcp_wmem control the minimum, default, and maximum sizes of TCP socket receive and send buffers. net.core.rmem_max and net.core.wmem_max set the system-wide maximum. For high-bandwidth, long-delay networks (research networks, transatlantic links), the default buffer sizes may be insufficient to keep the network pipe full. The bandwidth-delay product (BDP = bandwidth * RTT) defines how much data must be in flight to saturate a link; buffers must be at least this large. net.ipv4.tcp_timestamps enables RFC 7323 timestamps (used for RTT measurement and PAWS). net.ipv4.tcp_sack enables SACK. net.ipv4.tcp_window_scaling enables window scaling. net.ipv4.tcp_congestion_control selects the congestion control algorithm (options include cubic, bbr, reno, vegas). TCP BBR (Bottleneck Bandwidth and Round-trip propagation time), developed at Google and released in 2016, is a model-based congestion control algorithm that estimates the bottleneck bandwidth and minimum RTT rather than inferring congestion from packet loss, achieving significantly higher throughput

on lossy or high-latency networks.

Security considerations at the Transport layer include TCP SYN flood attacks (mitigated by SYN cookies), TCP RST injection attacks (where an attacker can terminate a TCP connection by spoofing RST segments with a valid sequence number — mitigated by RFC 5961, which adds a challenge ACK mechanism), and port scanning (attacker probes ports to discover open services — mitigated by firewalls). UDP amplification attacks exploit UDP services that return large responses to small requests (like DNS, NTP, and SSDP) to reflect amplified traffic at a victim; a spoofed UDP request from a victim's IP to an NTP server can return a response 100x larger, achieving high-volume DDoS. Source IP address validation (BCP 38, RFC 2827) would prevent spoofing-based attacks but is not universally deployed.

The port numbers used by an application are visible in tools such as netstat (or its modern replacement ss on Linux), which display all open sockets, their states (LISTEN, ESTABLISHED, TIME_WAIT, etc.), and their associated processes. The lsof command on Unix-like systems shows file descriptors, including sockets, associated with each process. tcpdump and Wireshark capture and analyze network packets, allowing examination of TCP handshakes, sequence numbers, window sizes, and flags in detail. These tools are indispensable for debugging network applications and understanding Transport layer behavior in practice.

Summary of Key Numbers and Facts

- OSI model has 7 layers; TCP/IP model has 4 layers. Transport layer is Layer 4 in OSI.
- Port numbers are 16-bit unsigned integers: range 0–65535.
- Well-known ports: 0–1023. Registered ports: 1024–49151. Ephemeral/dynamic ports: 49152–65535.
- Linux default ephemeral port range: 32768–60999.
- UDP header size: exactly 8 bytes (four 16-bit fields: source port, destination port, length, checksum).
- UDP maximum payload: 65507 bytes (65535 - 20 byte IP header - 8 byte UDP header).
- TCP header size: minimum 20 bytes, maximum 60 bytes (with up to 40 bytes of options).
- TCP segment header fields: source port (16 bits), destination port (16 bits), sequence number (32 bits), acknowledgment number (32 bits), data offset (4 bits), flags (9 bits: URG, ACK, PSH, RST, SYN, FIN, ECE, CWR, NS), window size (16 bits), checksum (16 bits), urgent pointer (16 bits).
- TCP three-way handshake: SYN → SYN-ACK → ACK. Connection establishment uses minimum 1 RTT.
- TCP connection termination: four segments — FIN, ACK, FIN, ACK (four-way). TIME_WAIT duration: 2 * MSL (typically 120 seconds).
- SRTT formula: SRTT = (7/8) * SRTT + (1/8) * RTT_sample (alpha = 1/8).
- RTTVAR formula: RTTVAR = (3/4) * RTTVAR + (1/4) * |SRTT - RTT_sample| (beta = 1/4).

- RTO formula: RTO = SRTT + max(G, 4 * RTTVAR). RFC 6298 minimum RTO: 1 second.
- Fast retransmit threshold: 3 duplicate ACKs.
- SACK option carries up to 4 block ranges per segment (RFC 2018).
- Nagle's algorithm buffers small sends until an MSS of data is ready or all outstanding data is ACKed (RFC 896). Disabled by TCP_NODELAY option.
- Window Scale option: shift factor 0–14, maximum effective window 65535 * 2^14 ≈ 1 GB (RFC 7323).
- Slow start initial window: 1 MSS (classic) or 10 MSS (RFC 6928, modern). Doubles cwnd each RTT until ssthresh.
- Congestion avoidance: increases cwnd by approximately 1 MSS per RTT (linear, AIMD).
- On loss detection (3 dup ACKs): ssthresh = max(cwnd/2, 2*MSS); Reno enters fast recovery.
- On timeout: ssthresh = max(cwnd/2, 2*MSS); cwnd reset to 1 MSS; restart slow start.
- QUIC runs on UDP (RFC 9000). HTTP/3 runs on QUIC (RFC 9114).
- QUIC reduces connection establishment to 1 RTT (0 RTT for resumption) vs. TCP + TLS 1.3 at 2 RTTs.
- Common well-known ports: FTP 20/21, SSH 22, Telnet 23, SMTP 25, DNS 53, DHCP 67/68, HTTP 80, POP3 110, NTP 123, IMAP 143, HTTPS 443, SMTPS 465, SNMP 161/162, LDAP 389, RDP 3389.
- TCP uses protocol number 6; UDP uses protocol number 17 in the IP header.
- TCP checksum and UDP checksum both computed using one's complement addition over a pseudo-header plus segment data.